

Database Schema Reference — Family Music Scrobbler PWA

Database Schema Reference - Family Music Scrobbler PWA

A complete reference for the PostgreSQL database schema used by the Family Music Scrobbler PWA. This document defines tables, relationships, constraints, indexes, and partitioning strategy for long-term scalable ingestion of Spotify listening history.

1. Overview

The database is designed for:

- 10+ years of historical scrobble storage
- Fast time-range queries
- Efficient ingestion deduplication
- Monthly partitioning for performance
- Secure token storage

All tables use **UUID primary keys**, **TIMESTAMPTZ** for time fields, and **B-Tree indexes** for user/time queries.

2. Core Tables

2.1 users

Stores local authentication users.

```
users
-----
id UUID PRIMARY KEY
email TEXT UNIQUE NOT NULL
display_name TEXT NOT NULL
leaderboard_opt_out BOOLEAN DEFAULT FALSE
created_at TIMESTAMPTZ DEFAULT NOW()
```

Indexes

- email (unique)
-

2.2 spotify_tokens

Stores encrypted Spotify refresh tokens.

```
spotify_tokens
-----
id UUID PRIMARY KEY
user_id UUID REFERENCES users(id) ON DELETE CASCADE
refresh_token_enc BYTEA NOT NULL
created_at TIMESTAMPTZ DEFAULT NOW()
updated_at TIMESTAMPTZ DEFAULT NOW()
```

Notes

- refresh_token_enc is AES-encrypted using Docker secrets.

2.3 tracks

Stores track metadata.

```
tracks
-----
id UUID PRIMARY KEY
name TEXT NOT NULL
artist TEXT NOT NULL
album TEXT
duration_ms INTEGER
spotify_id TEXT UNIQUE
created_at TIMESTAMPTZ DEFAULT NOW()
```

Indexes

- spotify_id (unique)

2.4 scrobbles (partitioned)

Stores individual plays. This is the largest table.

```
scrobbles (parent table)
-----
id UUID PRIMARY KEY
user_id UUID REFERENCES users(id) ON DELETE CASCADE
track_id UUID REFERENCES tracks(id)
spotify_play_id TEXT NOT NULL
played_at TIMESTAMPTZ NOT NULL
created_at TIMESTAMPTZ DEFAULT NOW()
```

Constraints

- `UNIQUE(user_id, spotify_play_id)` - strict deduplication

Indexes

- `(user_id, played_at)` - chronological queries
- `(spotify_play_id)` - ingestion dedupe

Partitioning

The parent table is partitioned by **RANGE (played_at)** into monthly partitions:

```
scrobbles_2026_09
scrobbles_2026_10
scrobbles_2026_11
...
```

Each partition contains:

```
CHECK (played_at >= 'YYYY-MM-01' AND played_at < 'YYYY-MM-01'::date + INTERVAL '1 month')
```

2.5 ingestion_state

Tracks ingestion worker progress.

```
ingestion_state
-----
id UUID PRIMARY KEY
user_id UUID REFERENCES users(id) ON DELETE CASCADE
last_polled TIMESTAMPTZ
poll_interval_min INTEGER DEFAULT 10
updated_at TIMESTAMPTZ DEFAULT NOW()
```

Notes

- Used by APScheduler worker.
- Ensures ingestion resumes correctly after restart.

3. Relationships Diagram (Textual)

```
users
├── spotify_tokens (1:1)
├── ingestion_state (1:1)
└── scrobbles (1:N)

tracks
└── scrobbles (1:N)
```

4. Security & Data Governance

Sensitive Fields

- refresh_token_enc (AES encrypted)
- No plaintext tokens stored

Cascading Deletes

Deleting a user cascades:

- scrobbles
- spotify_tokens
- ingestion_state

GDPR / Data Export

User data export includes:

- tracks
- scrobbles
- profile

5. Migration Strategy

Tools

- Alembic for schema migrations

Rules

- Never modify partition tables directly
- Always modify parent table and regenerate partitions if needed
- Partition creation automated monthly via migration script

6. Performance Considerations

Partitioning Benefits

- Faster queries for recent tracks
- Reduced index size

- Easier archival of old partitions

Indexing Strategy

- (user_id, played_at) supports dashboard queries
- (spotify_play_id) supports ingestion dedupe

Storage

- PostgreSQL volume stored on SSD for fast I/O
-

7. Summary

This schema is designed for:

- High-volume ingestion
- Long-term storage
- Fast analytics
- Secure token handling
- Predictable partitioning

It provides a robust foundation for the Family Music Scrobbler PWA backend and ingestion worker.